



EMEDIX JIFFY

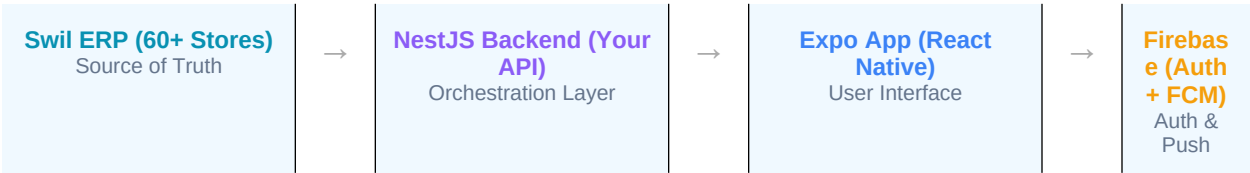
App Flow & ERP Integration Document

Backend Architecture Reference · Version 1.0
Stack: React Native Expo · NestJS · MySQL · Firebase · Swil ERP

1. System Architecture Overview

Emedix Jiffy is a hyperlocal medicine delivery app serving 60+ pharmacy stores across North India. The system follows a hub-and-spoke model where each store runs Swil ERP as the source of truth for inventory, pricing, and order management. The backend acts as an orchestration layer between the mobile app, ERP, payment gateway, and delivery partner.

Core Data Flow



Key Design Principles

- **ERP is always source of truth:** Stock availability, pricing, and order status always come from Swil ERP. Your backend never overrides these.
- **Post-confirmation payment:** Payment is initiated after cart validation, not before. No stock reservations until payment is confirmed.
- **Store-scoped sessions:** Every cart, product list, and price is tied to a specific store. Switching store or address resets the cart.
- **Idempotent ERP sync:** All polling jobs upsert by ERP codes. Re-running any sync job is always safe with no duplicates.
- **Stateful order tracking:** Orders follow a strict state machine: PENDING → CONFIRMED → PACKED → DISPATCHED → DELIVERED. Every transition is logged.

2. ERP API Integration (Swil)

Swil ERP will develop 5 APIs as confirmed. Your backend also maintains 3 inbound webhooks. Below is the complete integration map.

2.1 Swil's 5 APIs — What They Do

#	API Name	Direction	What Swil Returns	What Your Backend Does
1	Product	YOU → ERP	Full product/medicine catalog with ERP codes, name, category, MRP, unit, HSN	Full upsert into products table by <code>erp_product_code</code> . Run on first setup and weekly.
2	Product Update	YOU → ERP	Only products changed since a given timestamp (delta sync)	Upsert only changed records. Pass <code>last_synced_at</code> as query param. Runs every 15 mins.
3	Product Stock	YOU → ERP	Stock qty + store-level selling price per store	Upsert <code>store_inventory</code> table by (<code>store_id</code> , <code>product_id</code>). Run every 15 mins per store in parallel.
4	Order Download	YOU → ERP	Acknowledgement + <code>erp_order_id</code> after ERP creates the order	Called after payment success. Send items, qty, customer info, delivery address. Store <code>erp_order_id</code> .
5	Invoice Upload	ERP → YOU	Invoice PDF or URL + invoice number after billing is done	Store <code>invoice_url</code> and <code>invoice_number</code> on the order. Make available in-app and at delivery.

2.2 Your 3 Inbound Webhooks (Already Live)

These are endpoints on your backend that ERP calls to push real-time updates:

Webhook Endpoint	Triggered By ERP When	Your Action
------------------	-----------------------	-------------

2.3 ERP Polling Schedule

Job	Frequency	Notes
Product Update Sync	Every 15 mins	Delta only. Pass <code>last_synced_at</code> . Updates products table.
Store Inventory Sync	Every 15 mins	Parallel per store. 60+ concurrent calls. Updates <code>store_inventory</code> .
Active Order Status Poll	Every 2 mins	Only polls orders in CONFIRMED or PACKED state. Fallback if webhook misses.
Full Product Catalog Sync	Weekly / Manual	Admin-triggered or weekly. Full upsert of all products.
Invoice Fetch	On PACKED event	Triggered when order hits PACKED. Fetches invoice from ERP.

3. Complete App Flow

The following covers all 9 phases of the user journey from login to delivery, including every edge case and exception that must be handled in production.



PHASE 1 Authentication & Onboarding

- 1 User opens app → enters mobile number → OTP sent via Firebase Authentication
- 2 OTP verified → Firebase issues ID token → backend verifies token via Firebase Admin SDK
- 3 Backend checks if user exists by `firebase_uid` → creates new record or fetches existing
- 4 Backend issues internal JWT access token (15 min) + refresh token (30 days)
- 5 New user: prompt for name entry before proceeding to home screen



Edge Case: OTP timeout (60s): Show resend button with 30s cooldown. Max 3 resends, then block for 10 minutes.



Edge Case: Invalid/expired Firebase token: Return 401 to app. App clears local session and redirects to login screen.



Edge Case: Phone number already registered with different Firebase account: Merge by phone number, update `firebase_uid`.



PHASE 2 Location Detection & Store Assignment

- 1 App requests device location permission on first launch
- 2 If granted: use GPS coordinates. If denied: show mandatory manual address entry screen
- 3 User's coordinates sent to backend → Haversine distance calculated against all active stores
- 4 Nearest store within `delivery_radius_km` assigned as the session store

- 5 Store details, hours, and availability loaded and shown to user
- 6 User can manually switch store only if their address falls within that store's delivery radius (backend validates)
- 7 Address saved to user_addresses table with label (Home/Work/Other) and is_default flag



Edge Case: No store within delivery radius: Show 'Service not available in your area' screen. Block further flow.



Edge Case: Nearest store is closed (outside hours or holiday): Show next available store OR display 'Opens at HH:MM' with countdown.



Edge Case: User switches delivery address mid-session: Re-run store detection. If store changes, cart must be cleared (cart is store-scoped).



Edge Case: User tries to manually select a store outside their delivery area: Return 400 with 'Address not in delivery zone' message.



PHASE 3 Catalog Browsing & Product Discovery

- 1 App fetches categories and products for assigned store from your backend
- 2 Products filtered by store_inventory: is_available = true AND stock_qty > 0
- 3 Products display store-specific selling price from store_inventory table (not global MRP)
- 4 Full-text search runs against your DB (product name, salt composition, brand, category)
- 5 Medicines with requires_prescription = true shown with Rx badge — flagged throughout
- 6 Top-selling / featured products shown on home screen from a curated list or sales data



Edge Case: Product is in catalog but stock = 0 at this store: Show 'Out of Stock' badge. Cannot add to cart. Do not hide product.

 **Edge Case:** ERP sync is delayed (last_synced_at > 30 mins ago): Show 'Stock information may be slightly outdated' banner in-app. Alert admin via sync health logs.

 **Edge Case:** Product exists at another store but not this one: Do not show. Catalog is always store-scoped.



PHASE 4 Cart Management

- 1 User adds item → backend validates stock in real-time against store_inventory
- 2 Cart created for (user_id, store_id). Only one ACTIVE cart per user at any time.
- 3 Item quantity capped at available stock_qty (cannot add more than in stock)
- 4 Rx medicines in cart: user prompted to upload prescription before checkout
- 5 Cart stores unit_price snapshot at time of add (price shown to user is locked at this point)
- 6 Cart expiry: if idle for 2 hours, mark as EXPIRED. User starts fresh on next open.

 **Edge Case:** User has cart for Store A, then switches address triggering Store B: Clear Store A cart. Start new cart for Store B. Warn user before clearing.

 **Edge Case:** Item added to cart goes out of stock before checkout: Pre-checkout validation will catch this (see Phase 5).

 **Edge Case:** Prescription not uploaded for Rx item: Block 'Proceed to Checkout' button. Show 'Upload prescription to continue'.

 **Edge Case:** User tries to add more qty than stock: Cap at max stock. Show 'Only X left in stock' message.



PHASE 5 Pre-Checkout Validation

This is a critical safety gate. Run before showing the payment screen. Never skip.

- 1 Backend re-fetches store_inventory for every item in cart
- 2 Check 1 — Stock: If any item now has stock_qty = 0, return list of unavailable items to app
- 3 Check 2 — Price: If selling_price changed since cart was created, notify user with old vs new price
- 4 Check 3 — Store status: Confirm store is currently open and accepting orders
- 5 Check 4 — Prescription: Confirm all Rx items have valid prescription uploads
- 6 All checks pass → proceed to payment screen with final price breakdown



Edge Case: Items out of stock: App shows which items are unavailable. User must remove them to continue. Cannot checkout with 0-stock items.



Edge Case: Price changed: Show price change notification. User must explicitly confirm new price before proceeding.



Edge Case: Store closed at checkout time: Block checkout. Show 'Store is currently closed. Your cart is saved.'



PHASE 6 Payment

Payment gateway TBD (Razorpay / PhonePe recommended). Flow is gateway-agnostic.

- 1 Payment screen shows final amount: subtotal + delivery charge - discount
- 2 User selects payment method and completes payment on gateway UI
- 3 On payment SUCCESS: proceed immediately to order creation (Phase 7)

- 4 On payment FAILURE: return user to cart. Show failure reason. Do not create order.
- 5 Generate idempotency key before initiating payment. Use to prevent duplicate orders on retry.



Edge Case: Payment success but order creation API fails: Critical. Log to payment_reconciliation table. Retry order creation with same idempotency key. Alert admin. Do not show failure to user yet.



Edge Case: User closes app mid-payment: On next app open, check pending payment status with gateway. Reconcile before showing home screen.



Edge Case: Double tap / duplicate payment attempt: Idempotency key on order prevents duplicate order creation. Gateway-level deduplication for payment.



PHASE 7 Order Creation & ERP Push

- 1 Backend opens DB transaction: create order record (status: PENDING), mark cart as ORDERED
- 2 order_number generated: format EJ-YYYYMMDD-XXXX
- 3 Backend calls Swil Order Download API: sends items, quantities, customer phone, delivery address, order_number
- 4 ERP responds with erp_order_id → stored on order record → status updated to CONFIRMED
- 5 FCM push sent to user: 'Order #EJ-XXXXX confirmed! Store is preparing your medicines.'
- 6 order_status_logs entry created for every status transition



Edge Case: ERP Order Download API fails: Retry 3x with exponential backoff (2s, 4s, 8s). If all fail: order stays PENDING, alert admin via internal notification, do not show error to user.



Edge Case: Store rejects order from ERP (sudden stock issue): ERP pushes REJECTED status to your webhook → cancel order → initiate refund → send FCM to user with apology and refund ETA.



Edge Case: DB transaction fails mid-way: Rollback entirely. Payment already taken — log to reconciliation.

Admin resolves manually.



PHASE 8 Order Lifecycle & Real-Time Tracking

- 1 ERP webhook pushes status updates: CONFIRMED → PACKED → DISPATCHED → DELIVERED
- 2 Fallback: Backend polls ERP every 2 mins for all orders in CONFIRMED or PACKED state
- 3 Every status change → FCM push notification + order_status_logs entry
- 4 On PACKED: backend fetches invoice from ERP (API 5 or webhook). Stores invoice_url + invoice_number
- 5 On DISPATCHED: delivery partner assigned (TBD). Tracking info shown to user.
- 6 On DELIVERED: order closed, invoice accessible in-app, physical copy given at door by driver



Edge Case: Webhook missed (network issue): 2-min polling job serves as fallback. No status update is permanently missed.



Edge Case: Invoice fetch fails: Retry silently 3x. Do not block delivery flow. Invoice can be fetched on demand when user taps 'View Invoice'.



Edge Case: Order stuck in CONFIRMED for > 30 mins: Alert admin. Optionally notify user: 'Store is taking longer than usual'.



PHASE 9 Post-Order & Account Management

- 1 User can view complete order history with status, items, and invoice
- 2 Invoice downloadable as PDF from in-app (fetched from ERP via stored invoice_url)
- 3 Order cancellation allowed only in PENDING or CONFIRMED state — not after PACKED
- 4 Cancellation: sends cancellation to ERP → triggers refund if paid → FCM confirmation to user

- 5 User can reorder from previous orders (pre-fills cart with same items, validates stock fresh)



Edge Case: User tries to cancel a PACKED order: Block with message 'Order is already being packed and cannot be cancelled. Contact store.'



Edge Case: Reorder — items no longer available: Show which items are out of stock. Add only available items to cart.

4. Order State Machine

Every order must follow this exact state progression. No state can be skipped. Every transition must be logged in `order_status_logs` with timestamp, actor (user/erp/system/admin), and notes.

From State	To State	Triggered By	Action Required
—	PENDING	Payment success	Create order in DB, push to ERP
PENDING	CONFIRMED	ERP webhook / poll	Store <code>erp_order_id</code> , send FCM
CONFIRMED	PACKED	ERP webhook / poll	Fetch invoice, send FCM
PACKED	DISPATCHED	ERP webhook / Delivery API	Assign driver, send FCM with tracking
DISPATCHED	DELIVERED	ERP webhook / Driver confirm	Close order, send FCM with invoice
PENDING/ CONFIRMED	CANCELLED	User / ERP rejection	Initiate refund, send FCM
ANY	FAILED	System error	Log to reconciliation, alert admin

5. Notification Events (FCM)

Every significant event in the order journey triggers a Firebase Cloud Messaging push notification to the user's device.

Trigger Event	Notification Title	Notification Body
Order Confirmed	Order Confirmed! 	Your order #EJ-XXXXXX has been confirmed. The store is preparing your medicines.
Order Packed	Order Packed 	Your order is packed and ready. Delivery partner will be assigned shortly.
Order Dispatched	On the way! 	Your order is out for delivery. Expected arrival in X minutes.
Order Delivered	Delivered! 	Your order has been delivered. Tap to view your invoice.
Order Cancelled	Order Cancelled	Your order #EJ-XXXXXX has been cancelled. Refund will be processed within 3-5 days.
Payment Failed	Payment Failed	Your payment could not be processed. Please try again.
Out of Stock Alert	Item Unavailable	An item in your cart is now out of stock. Please review your cart.

— End of Document —